MODULE 04 · DOMAIN 1 - DESIGN APPLICATIONS

Tools and multi-stage reasoning

What a tool is, how to describe one, which order to call them in, and who decides.

SECTION 1 of 6 EXAM WEIGHT 14% MODULE 04 of 40

admin@sthithapragna.com

A tool turns a model that talks into a system that acts. **The model never runs the tool. It asks, and your code runs it.**

What happens during one tool call

01
Describe
Tool names, descriptions and parameters go to the model.

02
Request
The model returns a tool name and arguments.

03
Execute
Your application runs the tool.

04
Return
The result is added to the conversation.

05
Continue
The model calls another tool, or answers.

ON THE EXAM

The model chooses tools only from their descriptions. A vague description is the most common reason an agent picks the wrong tool.

Tools that gather, and tools that act

KIND	EXAMPLES	RISK
Gather knowledge	Search documents, query a table, call a live price API	Low. Reads only
Compute	Run code, calculate a premium, convert units	Low, if sandboxed
Take action	Create a ticket, issue a refund, send an email	High. Changes the world

ON THE EXAM

Gather before you act. And any action that cannot be undone deserves a check first: permissions, validation, or a person.

A tool the model will use correctly

- A **name** that says what it does: get_order_status, not helper2.
- A **description** saying when to use it, and when not to.
- **Typed parameters**, each with its own description and format.
- A **small, structured result**. Return what is needed, not the whole table.
- **One job per tool**, with no overlap between tools.

> ON THE EXAM
>
> If two tools could both answer the same request, the model will pick between them unpredictably. Make their descriptions exclusive.

The ReAct loop: reason, act, observe

01

Reason

What do I know, and what do I need next?

›

02

Act

Call the tool that gets it.

›

03

Observe

Read what came back.

›

04

Repeat or answer

Loop until the goal is met, or stop.

ON THE EXAM

ReAct is what the guide means by planning and adapting while reasoning. Pick it when later steps depend on earlier results.

Rules for the order of operations

- **Dependencies first.** You need the order ID before you can check its refund status.
- **Verify, then gather, then act.** Identity and eligibility come before any change.
- Cheap and fast before slow and costly, when either could settle it.
- Confirm before anything irreversible.
- Always set a **stop condition**, such as a maximum number of steps.

> ON THE EXAM
>
> When asked to order tools, find the irreversible action. It goes last, after everything that could prove it should not happen.

Pennant Outdoor processes a return

01
Verify
Confirm the customer's identity.

02
Look up
Fetch the order and delivery date.

03
Retrieve
Returns policy for that product type.

04
Decide
Eligible? Inside the window?

05
Act
Create the return, then confirm.

ON THE EXAM

Four gathering and checking steps protect one action. That ratio is normal.

Classify once, or loop until done?

A ROUTER

Decides once, hands off

- Request types are clearly separate
- Each type needs one handler
- Low latency, low cost
- Easy to test each route

AN AGENT

Decides at every step

- A request may need several tools
- The next step depends on a result
- Paths vary between requests
- Slower and costlier, but flexible

THE TIEBREAKER

If each request type is handled completely by one component, route. If handling one request needs a sequence that depends on what comes back, use an agent.

Is the answer a value, or a passage?

CALL A TOOL WHEN

The answer is a value in a system

- Stock level, price, balance, status
- Changes minute to minute
- Must be exact
- Lives in a table or an API

RETRIEVE WHEN

The answer is in prose

- Policies, manuals, contracts
- Changes occasionally
- Needs interpreting
- Lives in documents

THE TIEBREAKER

Putting live values into a vector index makes them stale and fuzzy. Putting documents behind a SQL tool makes them unsearchable.

Three ways to give an agent a tool

APPROACH	BEST FOR	TRADE-OFF
Unity Catalog function	Structured lookups where the query is known and the agent supplies parameters	Governed and discoverable. Needs serverless compute
Agent code tool	Logic defined directly in the agent's code	Fast and flexible. Less governance and discoverability
MCP server	Reusable tools, Databricks data, or external services	Standard interface. Managed, external or custom

ON THE EXAM

Wrapping http_request in a Unity Catalog function still works, but Databricks no longer recommends it for new external integrations.

Tools need guardrails of their own

- **Least privilege.** A tool can do only what its identity is allowed to do.
- **Human in the loop** for high-impact or irreversible actions.
- Validate arguments before executing. The model can invent an order number.
- Use a tool for arithmetic and lookups. Never trust the model's mental maths.
- Tool results are **data, not instructions**. Text inside them can try to steer the agent.

> ON THE EXAM
>
> An agent is only as safe as the most powerful tool it can call. Scope the tools, not just the prompt.

> A concert venue wants an assistant that answers questions about seat availability, which changes every few seconds in the ticketing system.
>
> It must also answer questions about venue rules, such as bag sizes and accessibility, which are described in a set of PDF guides.

How should the engineer design the assistant?

A Index hourly snapshots of seat availability alongside the PDF guides

B Fine-tune a model on past availability data and the venue rules

C Load current availability into the system prompt when the app starts

D Give the agent a live availability tool, plus retrieval over the PDF guides

◆ PAUSE AND DECIDE

How should the engineer design the assistant?

A Hourly snapshots in the index

An hour stale for data that changes every few seconds.

B Fine-tune on both

Past availability is useless for today's show. Rules would freeze.

C Availability in system prompt

Correct for a few seconds after start-up, then wrong.

D Live tool plus retrieval

Values from the system, passages from the documents.

THE SIGNAL

Changes every few seconds, and described in PDF guides. One value, one passage: a tool and a retriever.

> A refunds agent has four tools: issue_refund, get_order, check_refund_policy and verify_customer_identity.
>
> Refunds cannot be reversed once issued, and the policy differs by product category.

In which order should the agent use the tools?

A issue_refund, get_order, verify_customer_identity, check_refund_policy

B verify_customer_identity, get_order, check_refund_policy, issue_refund

C check_refund_policy, issue_refund, verify_customer_identity, get_order

D get_order, issue_refund, verify_customer_identity, check_refund_policy

◆ PAUSE AND DECIDE

In which order should the agent use the tools?

A **Refund first**

Acts before knowing anything. The irreversible step is first.

B **Verify, order, policy, refund**

Verify, gather, check, then act. The irreversible step is last.

C **Policy, refund, verify, order**

Checks policy without knowing the product, then refunds unverified.

D **Order, refund, verify, policy**

Refunds before identity or eligibility is known.

THE SIGNAL

Refunds cannot be reversed. Put the irreversible tool last, and everything that could stop it before.

> An internal support agent has two tools named search_kb and search_tickets. Both have the description: Searches data.
>
> The agent often searches old tickets when users ask how a product feature works, and misses the knowledge base article that answers it.

What is the most effective fix?

A Rewrite each tool's description to state what it contains and when to use it

B Raise the temperature so the agent explores both tools more often

C Merge both into a single tool that searches everything

D Move to a model with a larger context window

◆ PAUSE AND DECIDE

What is the most effective fix?

A Clear, exclusive descriptions

The model chooses only from descriptions. Give it the difference.

B Higher temperature

Random choice between tools is still wrong half the time.

C Merge into one tool

Hides the distinction instead of explaining it. Results mix sources.

D Larger context window

The model is not short of space. It is short of information.

THE SIGNAL

Both tools have the same description. The model cannot choose between two things it has been told are identical.

> An HR helpdesk receives three kinds of request: payslip questions, leave balance checks, and questions about HR policy. Each kind is fully handled by one existing component.
>
> Answers must come back in under two seconds, and the team wants each path tested separately.

Which design fits best?

A. An autonomous agent with all three components as tools, deciding step by step

B. A multi-agent supervisor coordinating three specialist agents

C. A classifier that routes each request to its one handler

D. One retrieval chain over all HR data, including payslips

◆ PAUSE AND DECIDE

Which design fits best?

A **Autonomous agent**

Works, but loops cost latency and nothing needs a second step.

B **Multi-agent supervisor**

Most complexity and latency. Nothing to coordinate.

C **Classifier router**

Decides once, hands off, fast, and each route is testable.

D **One retrieval chain**

Payslips and balances are values in systems, not passages.

THE SIGNAL

Each kind is fully handled by one existing component. When one handler completes each request, route; do not loop.

What to carry forward

- The model **requests** a tool from its description. Your code runs it.
- Good tools: clear name, when-to-use description, typed parameters, one job each.
- Order: **verify, gather, check, act**. The irreversible action goes last.
- Router when one handler completes each request. **ReAct agent** when steps depend on results.
- Value in a system: tool. Passage in a document: retrieval.

> ON THE EXAM
>
> Next: module 05 looks at Agent Bricks, where Databricks builds the agent for you.